

PLACIFY

AI-Powered Multi-Tenant Campus Placement Platform

Software Engineering Report & Documentation

Version: 1.0.0

Author: Aakash Srivastava

Date: July 14, 2026

Technology Stack: React 19, TypeScript, PostgreSQL, Gemini AI

Placify — AI-Powered Multi-Tenant Campus Placement Platform

Engineering Report and Software Documentation

Version: 1.0.0

Author: Aakash Srivastava

Date: July 14, 2026

Technology Stack: React 19, TypeScript, Vite, Tailwind CSS, Radix UI, InsForge (PostgreSQL, Auth, Storage), Google Gemini AI, CodeMirror 6, Judge0 CE, CloudConvert, Mammoth.js

Table of Contents

1. [Executive Summary](#1-executive-summary)
2. [Project Overview](#2-project-overview)
 - [Problem Statement](#problem-statement)
 - [Proposed Solution](#proposed-solution)
 - [Key Objectives](#key-objectives)
 - [Scope of Work](#scope-of-work)
3. [System Architecture](#3-system-architecture)
 - [High-Level Multi-Tenant Architecture](#high-level-multi-tenant-architecture)
 - [Frontend Architecture](#frontend-architecture)
 - [Backend Architecture](#backend-architecture)
 - [Database Security and Data Isolation (RLS)](#database-security-and-data-isolation-rls)
 - [Request Execution Flow](#request-execution-flow)
 - [Role Hierarchy](#role-hierarchy)
 - [Deployment Topology](#deployment-topology)
4. [Technology Stack Justification](#4-technology-stack-justification)
5. [Folder Structure Analysis](#5-folder-structure-analysis)
6. [Functional Features Documentation](#6-functional-features-documentation)
7. [User Roles and Permissions Matrix](#7-user-roles-and-permissions-matrix)
8. [Workflow Diagrams](#8-workflow-diagrams)
9. [Database Documentation](#9-database-documentation)
10. [API Endpoint Documentation](#10-api-endpoint-documentation)
11. [Authentication & Authorization](#11-authentication--authorization)

12. [AI Modules & Search Engine](#12-ai-modules--search-engine)
 13. [Document Converter Microservice Design](#13-document-converter-microservice-design)
 14. [Security Architecture](#14-security-architecture)
 15. [Performance Optimizations](#15-performance-optimizations)
 16. [Deployment and Environment Guide](#16-deployment-and-environment-guide)
 17. [Testing and Quality Assurance](#17-testing-and-quality-assurance)
 18. [Screenshots and UI Architecture](#18-screenshots-and-ui-architecture)
 19. [Future Enhancements & Roadmap](#19-future-enhancements--roadmap)
 20. [Conclusion](#20-conclusion)
-

1. Executive Summary

Traditional campus placement systems suffer from fragmented processes, manual evaluation, and a lack of data-driven match-making. Students practice coding on external sites, check their resumes on disconnected ATS scoring platforms, and fill out endless forms. Placement officers rely on spreadsheet filters to verify academic qualifications. Recruiters are overwhelmed by resumes that lack structured indexing.

Placify addresses these issues by uniting students, corporate recruiters, and placement admins within a secure, multi-tenant cloud application. Built on **InsForge BaaS** and powered by **Google Gemini AI** and **Judge0**, Placify integrates real-time eligibility checking, in-browser code compilation, automated ATS resume scoring, and a LinkedIn-style Candidate Search Engine. Data isolation is maintained through a combination of PostgreSQL Row Level Security (RLS) and client-side database proxy handlers.

2. Project Overview

Problem Statement

Campus recruitment involves multiple coordinates (academics, skill testing, resume builders, and job boards) that rarely communicate with one another:

- **Process Fragmentation:** Students bounce between separate portals to build resumes, test skills, and search for jobs.
- **Verification Latency:** Admins manually verify CGPA thresholds and backlog counts, creating human error risks.
- **Recruitment Friction:** Corporate sourcers cannot query candidate databases using natural language or filter candidates dynamically by academic criteria.

Proposed Solution

Placify establishes a unified multi-tenant placement application. Each university registers as an independent tenant. The portal provides dedicated dashboards for:

- 1. **Platform Owners:** Oversee university subscriptions, approve tenants, and monitor platform health.
- 2. **Organization Admins:** Customize university placement settings, verify student profiles, and approve recruiter access.
- 3. **Sub-Admins (Placement Officers):** Manage active placement drives, input DSA tracks, and review applicant queues.
- 4. **Students:** Build resumes, practice DSA, run sandboxed code, and apply to eligible jobs.
- 5. **Recruiters:** Post jobs, view applicants via rich interactive views, and search talent databases using AI.

Key Objectives

- Provide secure multi-tenant isolation at the database level.
- Automate applicant eligibility gatekeeping using strictly-cast mathematical types.
- Integrate an in-browser code compilation runner using sandboxed execution.
- Power recruiter workflows with natural language resume harvesting and search indexing.

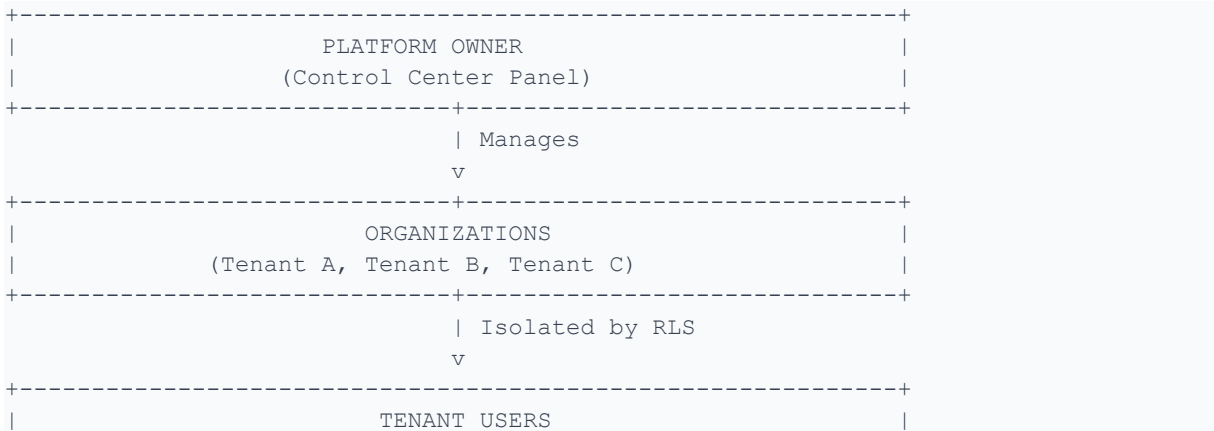
Scope of Work

The platform supports full profile management, resume upload, live ATS grading, a CodeMirror-based code simulator, a discussion forum, off-campus placement management, real-time alerts, and a multi-tenant administration backend.

3. System Architecture

High-Level Multi-Tenant Architecture

Placify uses a single-database, multi-tenant model. University organizations are stored in a central table, and operational tables map to them using an `organization_id` column.



```
| (Admins, Sub-Admins, Recruiters, Students) |
+-----+
|
```

Frontend Architecture

The client is a Single Page Application (SPA) built using React 19, TypeScript, and Vite. State is managed via specialized contexts (such as `RoleContext` and `ThemeContext`).

```
+-----+
| Vite React Frontend |
+-----+
| [AppRoutes] -> Guards (ProtectedRoute / RoleRoute) |
| [Contexts]   -> Authentication, Theme, Dynamic Roles |
| [Services]   -> Local Ollama Client, CloudConvert Fallback |
| [Lib]        -> InsForge Proxy Interceptor |
+-----+
| API Requests |
v
```

Backend Architecture

Placify uses a Backend-as-a-Service (BaaS) architecture powered by InsForge. The client communicates directly with PostgreSQL using the PostgREST REST API over HTTPS.

```
| v
+-----+
| InsForge BaaS Layer |
+-----+
| [PostgREST Engine] -> Automated REST Endpoint Router |
| [Auth Service]     -> JWT Sign-in / Verification |
| [Storage Engine]   -> AWS S3 Compatible File Buckets |
+-----+
| Database Operations |
v
```

Database Security and Data Isolation (RLS)

Security is enforced by PostgreSQL Row Level Security (RLS) and client-side proxies:

- PostgreSQL RLS Policies:** Restrict SELECT/INSERT/UPDATE/DELETE operations. The database calls `get_auth_user_organization_id()` to extract the tenant ID from the active JWT and match it against the row.
- Client-Side proxy (`insforge.ts`):** Wraps the database builder. If a queried table requires isolation, the proxy intercepts the call and appends `.eq('organization_id', orgId)` to queries. It also attaches the `organization_id` to insert payloads.

```
-- PostgreSQL Security Definer isolating tenant data
CREATE OR REPLACE FUNCTION public.get_auth_user_organization_id()
RETURNS UUID AS $$
DECLARE
    org_id UUID;
BEGIN
    SELECT organization_id INTO org_id FROM public.students WHERE user_id =
```

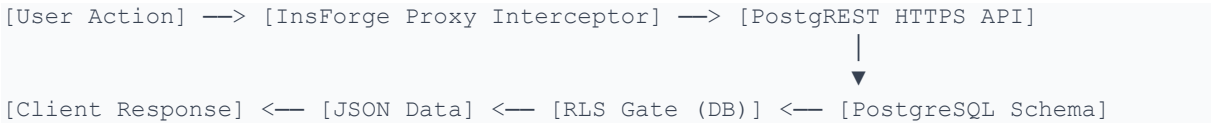
```
auth.uid()::text LIMIT 1;
    IF org_id IS NOT NULL THEN RETURN org_id; END IF;

    SELECT organization_id INTO org_id FROM public.recruiters WHERE user_id =
auth.uid()::text LIMIT 1;
    IF org_id IS NOT NULL THEN RETURN org_id; END IF;

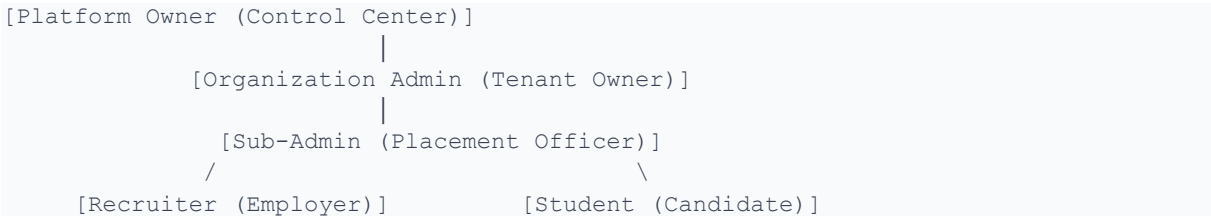
    SELECT organization_id INTO org_id FROM public.admins WHERE user_id =
auth.uid()::text LIMIT 1;
    IF org_id IS NOT NULL THEN RETURN org_id; END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql STABLE SECURITY DEFINER;
```

Request Execution Flow



Role Hierarchy



Deployment Topology

- **Frontend Hosting:** Vercel (Edge routing, serverless rewrites).
- **Database & API Services:** InsForge cloud platform.
- **Code Sandbox:** Public Judge0 cloud endpoints.

4. Technology Stack Justification

Layer	Technology	Selection Justification
Frontend Framework	React 19	Declarative UI development, concurrent rendering, and native compatibility with modern hooks.
Build Tooling	Vite	Rapid Hot Module

		Replacement (HMR) and roll-up builds for deployment.
Programming Language	TypeScript	Strong typing, compilation checks, and structure definitions.
CSS System	Tailwind CSS	Utility-first styling that keeps CSS bundles small and supports rapid design iteration.
BaaS Provider	InsForge	Combines authentication, serverless functions, file storage, and PostgreSQL databases into one unified platform.
Database	PostgreSQL	Supports relational schemas, transactional integrity, and nested <code>JSONB</code> formats.
Code Editor	CodeMirror 6	Lightweight alternative to Monaco Editor that reduces bundles by ~1.5MB.
AI Integration	Google Gemini AI	<code>gemini-2.5-flash</code> model for structured JSON output, resume analysis, and search query parsing.
Code Execution	Judge0 API	Secure sandboxing for evaluating multi-language compiler runs.

5. Folder Structure Analysis

placify/	
├── control-center/	# Platform Owner Admin Panel (Sub-project)
│ ├── src/	
│ │ ├── App.tsx	# Main administrative panel logic (258 KB)
│ │ └── index.css	# Control Center styling configurations
│ └── package.json	# Control Center package dependencies
├── database/	# Relational Database Migration files
│ ├── 00_create_base_schema.sql	# Base table setups
│ ├── 09_super_admin_schema.sql	# Super admin schema migrations
│ ├── 10_multi_tenant_schema.sql	# Multi-tenant organization additions
│ ├── 11_org_admin_schema.sql	# Sync triggers between admin tables
│ ├── 12_multi_tenant_rls.sql	# Security functions and RLS policies
│ └── placify_dump.sql	# Consolidated schema backups
├── docs/	# Engineering documents and backups
├── public/	# Static assets, logos, and images
├── src/	# Main Placify Application Directory
│ ├── components/	# Global components (buttons, inputs,
loaders)	
│ ├── context/	# State managers (RoleContext, ThemeContext)
│ ├── lib/	# Client configs (insforge.ts proxy client)
│ ├── modules/	# Role-based feature folders
│ │ ├── organization-admin/	# Org Admin settings and analytics dashboards
│ │ ├── recruiter/	# Recruiter job postings and candidate lists
│ │ └── student/	# Student portfolios, ATS checker, and
simulator	
│ └── subadmin/	# Placement cell applicant and verification
views	
│ ├── pages/	# Public pages (Auth, Landing, Role
Selection)	
│ ├── routes/	# Protected path configurations
│ └── services/	# Business services (notification, Ollama
API)	
│ └── utils/	# Eligibility validation and date automation
├── package.json	# Build scripts and project dependencies
└── vite.config.ts	# Build pipeline settings

6. Functional Features Documentation

Authentication

- **Custom Login Page:** Fully custom login page located at `/auth`. Implements `signInWithPassword()` and `signUp()` via the InsForge SDK client.
- **OTP Awaiting Gate:** Directs unverified recruits or students to an OTP screen if the account is marked `Pending`.

Role Management & Selection

- **Identity Switching:** If an email is linked to multiple profile types, users can select their active role on the `/select-role` screen.

- **Access Enforcement:** The system maps the active session to database roles and blocks unauthorized paths via route guards.

Organization Management & Approvals

- **Onboarding Queue:** Universities submit registration requests via `/auth`. These requests are queued in the `organization_requests` table.
- **Platform Approvals:** Platform owners review documentation, assign active codes, and provision tenant records.

Control Center & Settings

- **System Controls:** Platform owners can toggle global maintenance mode, change SMTP credentials, and manage max job limits.

Subscription System

- **Billing Access:** Platform owners can extend organization accounts, suspend access, and update statuses to `Trial`, `Active`, or `Expired`.

Student Portal & Profile Management

- **Portfolio Forms:** Students can upload profile images, link repositories, and list their projects, skills, and certifications.
- **Academic Verification:** Verification statuses block unqualified applications until approved by a sub-admin.

Recruiter Portal

- **Job Creation:** Recruiters post positions detailing branch criteria, CTCs, description guidelines, and application deadlines.
- **Hiring Views:** Recruiters can view applicant forms, download resumes, and update application statuses.

Sub-Admin (Placement Officers)

- **Review Pipeline:** Sub-admins verify student credentials, approve job listings, and manage DSA question sheets.

ATS Checker

- **Scoring Breakdown:** Analyzes resume templates on keyword match, action verbs, and layout formatting. Returns a score out of 100 with improvement advice.

Resume Builder

- **Template Exporters:** A text editor that exports resume structures to standard formats.
- **Document Conversions:** Converts PDF uploads to text locally via `pdfjs-dist`. DOCX uploads use `mammoth.js`.

Monaco Code Simulator

- **Sandboxed Engine:** A lightweight CodeMirror editor that compiles C++, Java, Python, and JavaScript code. It sends payloads to Judge0 to run test cases.

DSA Sheets

- **Category Trackers:** Provides curated question lists (such as LeetCode 75) mapped to top tech companies. Tracks student progress dynamically.

Community Forum

- **Forums & Threads:** Users can create categories, post discussions, comment, and upvote content.

Analytics

- **Dashboard Visualizations:** Renders placement trends, branch statistics, and applicant funnels using Recharts.

Notifications & Audit Logs

- **Alert Routing:** Notifies users of status updates and new jobs. Log tables audit administrative updates.

7. User Roles and Permissions Matrix

Operations / Features	Platform Owner	Org Admin	Sub-Admin	Recruiter	Student
Manage Tenants	Yes	No	No	No	No
Configure Settings	Yes	Yes	No	No	No
Approve Recruiters	No	Yes	Yes	No	No
Verify Students	No	Yes	Yes	No	No
Post Job Drives	No	No	Yes	Yes	No

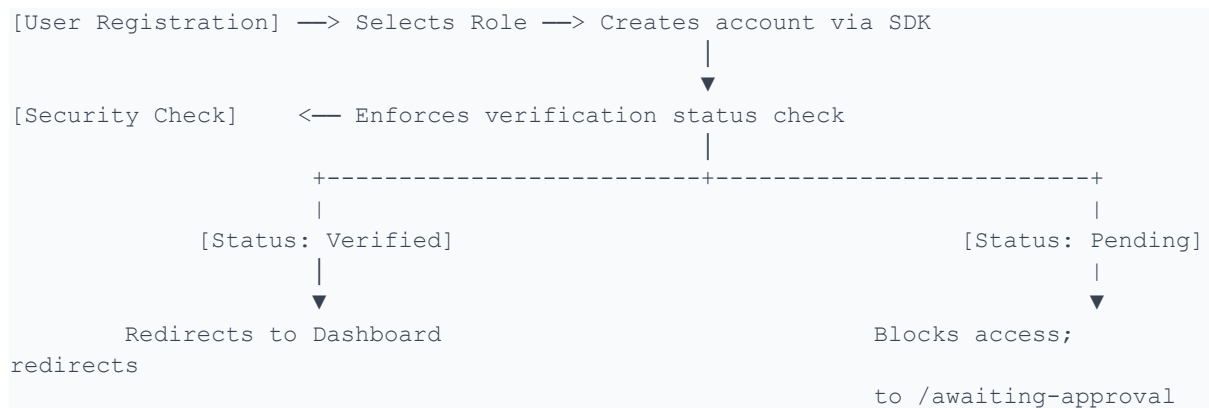
Manage DSA Sheets	No	No	Yes	No	No
Practice Coding	No	No	No	No	Yes
Apply for Jobs	No	No	No	No	Yes
AI Candidate Search	No	No	Yes	Yes	No

8. Workflow Diagrams

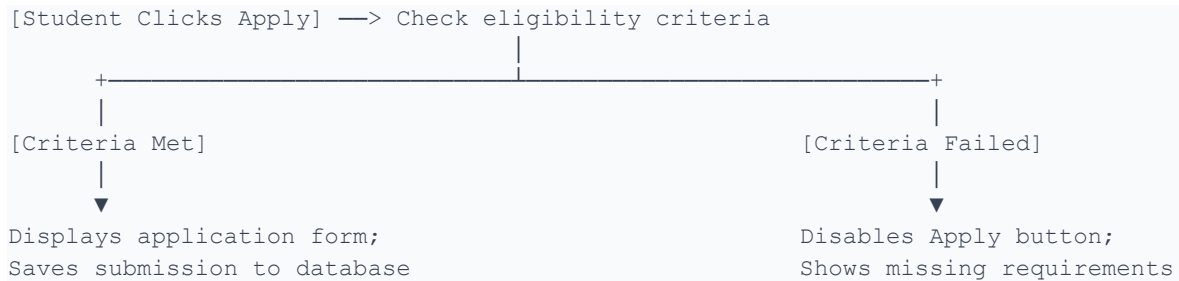
Organization Approval Flow



User Registration Flow



Job Application Eligibility Gate



9. Database Documentation

Here is the database schema for the 21 primary tables:

1. `organizations`

Stores tenant information.

- **Columns:**
- `id` (UUID, Primary Key, Default: `gen_random_uuid()`)
- `name` (TEXT, NOT NULL)
- `code` (TEXT, UNIQUE, NOT NULL)
- `website` (TEXT)
- `logo_url` (TEXT), `logo_key` (TEXT)
- `address` (TEXT)
- `status` (TEXT, Default: 'Active', Constraint: IN ('Active', 'Suspended'))
- `created_at` (TIMESTAMPTZ, Default: `now()`)
- `admin_id` (TEXT)

2. `students`

Stores student profiles.

- **Columns:**
- `id` (UUID, Primary Key)
- `user_id` (TEXT, UNIQUE, NOT NULL)
- `name` (TEXT, NOT NULL), `email` (TEXT, NOT NULL)
- `phone` (TEXT), `branch` (TEXT)
- `current_year` (INTEGER), `graduation_year` (INTEGER)
- `cgpa` (NUMERIC, Default: 0), `backlogs` (INTEGER, Default: 0)
- `placement_status` (TEXT, Default: 'not_placed')
- `resume_url` (TEXT), `resume_key` (TEXT)
- `profile_photo_url` (TEXT), `profile_photo_key` (TEXT)
- `bio` (TEXT)
- `linkedin_url` (TEXT), `github_url` (TEXT), `portfolio_url` (TEXT)

- `skills` (TEXT[])
- `status` (TEXT, Default: 'pending', Constraint: IN ('pending', 'verified', 'rejected', 'suspended'))
- `organization_id` (UUID, Foreign Key -> `organizations.id`)
- `college_id` (TEXT)
- **Constraints:**
- `UNIQUE` (`organization_id`, `college_id`)

3. `student\_skills`

Stores candidate skill tags.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, Foreign Key -> `students.id` ON DELETE CASCADE)
- `skill` (TEXT, NOT NULL)

4. `student\_projects`

Stores student portfolio projects.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, Foreign Key -> `students.id` ON DELETE CASCADE)
- `title` (TEXT, NOT NULL), `description` (TEXT)
- `technologies` (TEXT[])
- `project_url` (TEXT), `github_url` (TEXT)

5. `student\_certificates`

Stores student professional credentials.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, Foreign Key -> `students.id` ON DELETE CASCADE)
- `name` (TEXT, NOT NULL), `issuer` (TEXT)
- `issue_date` (DATE), `expiration_date` (DATE)
- `credential_id` (TEXT), `credential_url` (TEXT)

6. `student\_ai\_profiles`

Caches AI extraction metadata.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, UNIQUE, Foreign Key -> `students.id` ON DELETE CASCADE)
- `tenth_percentage` (NUMERIC), `twelfth_percentage` (NUMERIC)
- `internships_count` (INTEGER), `experience_months` (INTEGER)
- `certificates_names` (TEXT[])

- `extracted_skills` (TEXT[]), `extracted_technologies` (TEXT[]), `extracted_keywords` (TEXT[])
- `ai_tags` (TEXT[])
- `experience_level` (TEXT)
- `resume_summary` (TEXT)

7. `organization\_admins`

Stores tenant administrator accounts.

- **Columns:**
- `id` (UUID, Primary Key)
- `user_id` (TEXT, UNIQUE, NOT NULL)
- `organization_id` (UUID, Foreign Key -> `organizations.id` ON DELETE CASCADE)
- `name` (TEXT, NOT NULL), `email` (TEXT, NOT NULL)
- `is_active` (BOOLEAN, Default: `true`)

8. `subadmins`

Stores sub-admin accounts.

- **Columns:**
- `id` (UUID, Primary Key)
- `user_id` (TEXT, UNIQUE, NOT NULL)
- `organization_id` (UUID, Foreign Key -> `organizations.id` ON DELETE CASCADE)
- `created_by_admin` (UUID, Foreign Key -> `organization_admins.id`)
- `name` (TEXT, NOT NULL), `email` (TEXT, NOT NULL)
- `department` (TEXT), `role` (TEXT, Default: 'Placement Officer')
- `status` (TEXT, Default: 'Active', Constraint: IN ('Active', 'Suspended'))

9. `admins` (Legacy Alignment Table)

Synchronizes admins and sub-admins via database triggers.

- **Columns:**
- `id` (UUID, Primary Key)
- `user_id` (TEXT, UNIQUE, NOT NULL)
- `name` (TEXT, NOT NULL), `email` (TEXT, NOT NULL)
- `status` (TEXT, Default: 'Pending')
- `organization_id` (UUID, Foreign Key -> `organizations.id`)
- `role` (TEXT, Constraint: IN ('organization_admin', 'sub_admin'))
- `permissions` (TEXT[])

10. `recruiters`

Stores recruiter profiles.

- **Columns:**
- `id` (UUID, Primary Key)
- `user_id` (TEXT, UNIQUE, NOT NULL)

- `name` (TEXT, NOT NULL), `email` (TEXT, NOT NULL)
- `company` (TEXT)
- `status` (TEXT, Default: 'Pending')
- `organization_id` (UUID, Foreign Key -> `organizations.id`)
- `verification_status` (TEXT, Default: 'Pending')

11. `jobs`

Stores job postings.

- **Columns:**
- `id` (UUID, Primary Key)
- `role` (TEXT), `company` (TEXT, NOT NULL), `title` (TEXT, NOT NULL)
- `description` (TEXT), `job_type` (TEXT), `work_mode` (TEXT)
- `ctc` (NUMERIC), `stipend` (NUMERIC), `location` (TEXT[])
- `min_cgpa` (NUMERIC, Default: 0), `max_backlogs` (INTEGER, Default: 0)
- `allowed_branches` (TEXT[]), `allowed_years` (TEXT[])
- `application_deadline` (TIMESTAMPTZ)
- `num_rounds` (INTEGER), `interview_process` (TEXT)
- `required_skills` (TEXT[]), `tech_stack` (TEXT[])
- `status` (TEXT, Default: 'active')
- `logo_url` (TEXT)
- `organization_id` (UUID, Foreign Key -> `organizations.id`)

12. `job\_applications`

Tracks job applications.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, Foreign Key -> `students.id` ON DELETE CASCADE)
- `job_id` (UUID, Foreign Key -> `jobs.id` ON DELETE CASCADE)
- `status` (TEXT, Default: 'pending', Constraint: IN ('pending', 'under_review', 'shortlisted', 'rejected', 'accepted'))
- `application_form` (JSONB)
- `organization_id` (UUID, Foreign Key -> `organizations.id`)
- **Constraints:**
- `UNIQUE` (`student_id`, `job_id`)

13. `saved\_jobs`

Tracks bookmarked jobs.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, Foreign Key -> `students.id` ON DELETE CASCADE)
- `job_id` (UUID, Foreign Key -> `jobs.id` ON DELETE CASCADE)

- `organization_id` (UUID, Foreign Key -> `organizations.id`)
- **Constraints:**
- `UNIQUE (student_id, job_id)`

14. `coding\_problems`

Stores simulator code puzzles.

- **Columns:**
- `id` (UUID, Primary Key)
- `title` (TEXT, NOT NULL), `description` (TEXT, NOT NULL)
- `difficulty` (TEXT, Constraint: `IN ('Easy', 'Medium', 'Hard')`)
- `starter_code` (JSONB), `test_cases` (JSONB)
- `points` (INTEGER, Default: 0)

15. `coding\_submissions`

Tracks student code executions.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, Foreign Key -> `students.id` ON DELETE CASCADE)
- `problem_id` (UUID, Foreign Key -> `coding_problems.id` ON DELETE CASCADE)
- `code` (TEXT, NOT NULL), `language` (TEXT, NOT NULL)
- `status` (TEXT, NOT NULL)
- `points_earned` (INTEGER, Default: 0)
- `organization_id` (UUID, Foreign Key -> `organizations.id`)

16. `dsa\_companies`

Stores top tech tags for DSA questions.

- **Columns:**
- `id` (UUID, Primary Key)
- `name` (TEXT, UNIQUE, NOT NULL)

17. `dsa\_questions`

Stores DSA curriculum questions.

- **Columns:**
- `id` (UUID, Primary Key)
- `title` (TEXT, NOT NULL), `leetcode_url` (TEXT, NOT NULL)
- `difficulty` (TEXT, Constraint: `IN ('Easy', 'Medium', 'Hard')`)
- `category` (TEXT)
- `company_id` (UUID, Foreign Key -> `dsa_companies.id`)
- `organization_id` (UUID, Foreign Key -> `organizations.id`)

18. `dsa\_progress`

Tracks student question completion status.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, Foreign Key -> `students.id` ON DELETE CASCADE)
- `question_id` (UUID, Foreign Key -> `dsa_questions.id` ON DELETE CASCADE)
- `status` (TEXT, Default: 'solved', Constraint: IN ('solved', 'unsolved'))
- `organization_id` (UUID, Foreign Key -> `organizations.id`)
- **Constraints:**
- `UNIQUE (student_id, question_id)`

19. `ats\_scans`

Stores resume scans and AI feedback history.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, Foreign Key -> `students.id` ON DELETE CASCADE)
- `score` (INTEGER, NOT NULL)
- `feedback` (JSONB)
- `organization_id` (UUID, Foreign Key -> `organizations.id`)

20. `alumni`

Stores alumni details.

- **Columns:**
- `id` (UUID, Primary Key)
- `name` (TEXT, NOT NULL), `email` (TEXT, UNIQUE, NOT NULL)
- `company` (TEXT), `position` (TEXT), `branch` (TEXT)
- `graduation_year` (INTEGER)
- `available_for_mentorship` (BOOLEAN, Default: true)
- `linkedin_url` (TEXT)
- `organization_id` (UUID, Foreign Key -> `organizations.id`)

21. `referral\_requests`

Tracks student referral requests sent to alumni.

- **Columns:**
- `id` (UUID, Primary Key)
- `student_id` (UUID, Foreign Key -> `students.id` ON DELETE CASCADE)
- `alumni_id` (UUID, Foreign Key -> `alumni.id` ON DELETE CASCADE)
- `status` (TEXT, Default: 'pending', Constraint: IN ('pending', 'approved', 'rejected'))
- `message` (TEXT)
- `organization_id` (UUID, Foreign Key -> `organizations.id`)

- **Constraints:**
- `UNIQUE (student_id, alumni_id)`

22. `organization\_requests`

Manages the university onboarding queue.

- **Columns:**
- `id` (UUID, Primary Key)
- `organization_name` (TEXT, NOT NULL)
- `generated_org_code` (TEXT, UNIQUE, NOT NULL)
- `organization_email` (TEXT, UNIQUE, NOT NULL)
- `admin_email` (TEXT, UNIQUE, NOT NULL)
- `status` (TEXT, Default: 'Pending', Constraint: IN ('Pending', 'Under Review', 'Approved', 'Rejected', 'Need More Information'))

23. `audit\_logs`

Logs administrative actions.

- **Columns:**
- `id` (UUID, Primary Key)
- `performed_by` (TEXT, NOT NULL), `action` (TEXT, NOT NULL)
- `affected_user` (TEXT), `created_at` (TIMESTAMPTZ)
- `organization_id` (UUID, Foreign Key -> `organizations.id`)

24. `platform\_settings`

Stores platform-wide configuration settings.

- **Columns:**
- `key` (TEXT, Primary Key)
- `value` (JSONB, NOT NULL)

Entity Relationship (ER) Diagram

Below is the relational entity map in Mermaid format:

```
erDiagram
    organizations ||--o{ students : "hosts"
    organizations ||--o{ recruiters : "approves"
    organizations ||--o{ admins : "employs"
    organizations ||--o{ jobs : "organizes"

    students ||--o{ student_skills : "has"
    students ||--o{ student_projects : "builds"
    students ||--o{ student_certificates : "achieves"
    students ||--o{ student_ai_profiles : "analyzed_into"
    students ||--o{ job_applications : "submits"
    students ||--o{ saved_jobs : "bookmarks"
    students ||--o{ coding_submissions : "codes"
```

```
students |--o{ dsa_progress : "completes"
students |--o{ ats_scans : "scans"
students |--o{ referral_requests : "requests"

jobs |--o{ job_applications : "receives"
jobs |--o{ saved_jobs : "saved_by"
coding_problems |--o{ coding_submissions : "evaluates"
dsa_questions |--o{ dsa_progress : "tracked_in"
dsa_companies |--o{ dsa_questions : "tags"
alumni |--o{ referral_requests : "reviews"
```

10. API Endpoint Documentation

Placify uses **PostgREST**, which exposes database tables as REST endpoints. The API base URL is:

`https://39s3r2sh.ap-southeast.insforge.app`

Endpoint Specifications

1. **`GET /students`**

Fetches student profiles.

- **Parameters:**
- `select` (string): Fields to return.
- `user_id` (string): Match user ID.
- **Authentication Required:** Yes.
- **Response (200 OK):**

```
[
  {
    "id": "7a8b9c0d-1e2f-3a4b-5c6d-7e8f9a0b1c2d",
    "name": "Jane Doe",
    "email": "jane@university.edu",
    "branch": "CSE",
    "cgpa": 9.2
  }
]
```

2. **`POST /job\_applications`**

Submits a job application.

- **Authentication Required:** Yes.
- **Request Body:**

```
[
  {
    "student_id": "7a8b9c0d-1e2f-3a4b-5c6d-7e8f9a0b1c2d",
    "job_id": "0f1e2d3c-4b5a-6e7d-8c9b-0a1b2c3d4e5f",
    "application_form": {
      "phone": "9876543210",
      "graduationYear": 2027
    }
  }
]
```

```
}  
]
```

- **Response (201 Created):**

```
[  
  {  
    "id": "9f8e7d6c-5b4a-3e2d-1c0b-9a8b7c6d5e4f",  
    "status": "pending"  
  }  
]
```

3. ***PATCH /jobs***

Updates a job description.

- **Parameters:**
- `id` (string): Match job ID.
- **Authentication Required:** Yes (Recruiter/Admin only).
- **Request Body:**

```
{  
  "status": "closed"  
}
```

- **Response (200 OK):**

```
[  
  {  
    "id": "0f1e2d3c-4b5a-6e7d-8c9b-0a1b2c3d4e5f",  
    "status": "closed"  
  }  
]
```

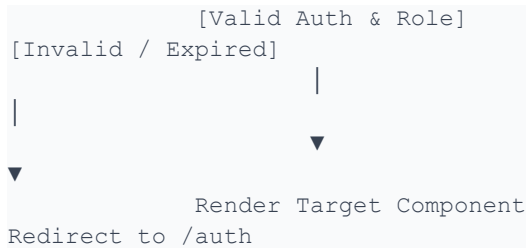
11. Authentication & Authorization

Custom Auth Architecture

Placify uses JWT-based authentication. Sessions are validated on the client side using the `RoleContext` and protected guards:

1. **Identity Resolution:** Pings `/api/auth/current` to retrieve user credentials.
2. **Guards:** Intercepts route navigation using `ProtectedRoute` and `RoleRoute`. If the user lacks the required role, they are redirected to `/select-role`.

```
[User Action] —> [AppRoutes.tsx] —> [ProtectedRoute] —> Checks RoleContext  
|  
+-----+  
|  
|
```

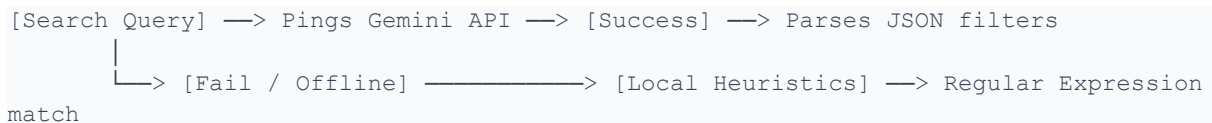


12. AI Modules & Search Engine

Query Parser & Fallback Pipeline

The AI Candidate Search Engine translates natural language queries into structured JSON database filters:

1. **AI Mapping:** Pings Google Gemini API to translate the query into structured JSON filters.
2. **Local Fallback Heuristics:** If the API rate limits (HTTP 429) or is offline, the system runs local regular expressions to parse search limits and keywords.



The Search Index Hack

Resumes are uploaded to storage, and their raw text content is parsed. The system appends this text to the `resume_summary` column in `student_ai_profiles` as a search tag (`[FULL_RESUME_TEXT_FOR_SEARCH_ENGINE]: ...`), making the entire resume searchable.

13. Document Converter Microservice Design

To provide high-fidelity file conversions without relying on third-party SaaS limits, we design a dedicated Document Converter microservice.

Architecture Topology



FastAPI Worker Code (`main.py`)

```
import subprocess
from fastapi import FastAPI, UploadFile, File
```

```

from fastapi.responses import FileResponse
import shutil
import os

app = FastAPI(title="Placify File Conversion Engine")

@app.post("/convert/docx-to-pdf")
async def convert_docx_to_pdf(file: UploadFile = File(...)):
    temp_docx = f"temp_{file.filename}"
    with open(temp_docx, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    # Run LibreOffice headless conversion
    cmd = ["soffice", "--headless", "--convert-to", "pdf", temp_docx]
    subprocess.run(cmd, check=True)

    pdf_name = temp_docx.replace(".docx", ".pdf")
    return FileResponse(pdf_name, media_type="application/pdf", filename=pdf_name)

```

14. Security Architecture

- **Database Isolation:** Row Level Security (RLS) restricts access based on `organization_id`.
- **Role Protection:** Routes are protected by client-side guards and database-enforced policies.
- **Input Sanitization:** Casting input fields (like CGPA) to float types prevents query injection.
- **CORS Configuration:** Configured to restrict API access to trusted domains.
- **Storage Access:** Files are stored in secure buckets, and access is managed via time-limited URLs.

15. Performance Optimizations

- **Lazy Loading:** Components and layouts are loaded lazily using React Suspense to reduce initial bundle sizes.
- **Response Caching:** AI scan outputs are cached in localStorage using SHA-256 hashes of the inputs to reduce API token usage.
- **Code Sandbox optimizations:** Replaced the heavy Monaco Editor package with CodeMirror 6, reducing the client bundle size by ~1.5MB.
- **Paginated DB Queries:** Uses PostgREST range headers (Range: 0-14) to fetch data in small batches.

16. Deployment and Environment Guide

Environment Configuration (`.env.production`)

```

VITE_INSFORGE_BASE_URL=https://39s3r2sh.ap-southeast.insforge.app
VITE_INSFORGE_ANON_KEY=your_production_anon_key
VITE_GEMINI_API_KEY=your_gemini_api_key
VITE_OLLAMA_URL=http://localhost:11434

```

```
VITE_OLLAMA_MODEL=llama3.2
VITE_USE_AI_ATS=true
```

Multi-Stage Dockerfile (`Dockerfile`)

```
# Stage 1: Build React Client
FROM node:25-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Serve via Nginx
FROM nginx:stable-alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

17. Testing and Quality Assurance

- **Manual Testing:** Admins manually verify onboarding, recruiter registrations, and student status changes.
- **Eligibility Verification:** Tested against invalid CGPA entries (e.g., matching string characters to numeric fields).
- **Role Routing Tests:** Verified that students are blocked from admin routes, and recruiters are restricted from student views.

18. Screenshots and UI Architecture

1. Unified Authentication (`/auth`)

- **UI Layout:** Dual-grid layout featuring a dark corporate side-panel and an interactive login form.
- **UI Components:** Radix Tab primitives that switch between Login, Student Signup, and Recruiter Registration modes.

2. Student Dashboard (`/student/dashboard`)

- **UI Layout:** A multi-widget dashboard displaying job application statistics, active drives, and upcoming tasks.
- **UI Components:** Displays resume scores and DSA progress using Recharts radial bars.

3. Recruiter Job Creator (`/admin/post-job`)

- **UI Layout:** A step-by-step wizard for posting job details, eligibility criteria, and selection rounds.
- **UI Components:** Multi-select search dropdowns for selecting allowed branches.

4. Monaco Code Simulator (`/code-simulator`)

- **UI Layout:** Split-screen layout displaying the problem description on the left and the code editor on the right.
 - **UI Components:** CodeMirror 6 editor with themes, run triggers, and standard output panels.
-

19. Future Enhancements & Roadmap

- **AI Assessment Engine:** Conduct mock video interviews using WebRTC and generate behavioral reports.
 - **OAuth Integrations:** Support single sign-on (SSO) using Google Workspace and GitHub.
 - **Advanced RLS Policies:** Define comprehensive RLS policies to restrict database mutations to profile owners.
 - **Dynamic Assessments:** Allow recruiters to generate custom coding tests for specific job postings.
-

20. Conclusion

Placify transforms campus recruitment by combining multi-tenant database design with AI-driven workflows. By automating eligibility verification, integrating in-browser code compilation, and enabling natural language candidate searches, it reduces recruitment friction. The resulting platform is a modern, scalable, and secure application ready for deployment.